

Stoquify Agent Runtime

Practical Execution Plan

Generated from executed prompt | 2026-07-22

Generated: 2026-07-22

Source prompt: C:\Users\J

COMPUTER\codex\attachments\le89edd20-892f-4267-b8b6-e94f63fb6c15\pasted-text.txt

Source roadmap:

docs/agents-runtime/STOQUIFY_AGENT_RUNTIME_STRATEGIC_ROADMAP_REPORT_2026-07-21.md

Scope: practical execution plan only. No application code, schema, dependency, or migration changes were made.

1. Execution Objective

This plan turns the Stoquify Agent Runtime Strategic Roadmap into a practical build program. The objective is to make Stoquify an evidence-backed operating copilot through a controlled, phased implementation that starts with a safe shared runtime, then adds read-only and read-and-draft agents.

The execution program must produce:

- one shared agent runtime, not one framework per module
- service-owned read tools first
- tenant, RBAC, module entitlement, redaction, freshness, and evidence controls before model output
- a read-only Command Agent internal pilot
- Cash/Reconciliation and Inventory/Replenishment agents in read-and-draft mode later
- approval, idempotency, observability, and low-risk controlled execution only after the runtime proves safe

The plan must not create a generic chatbot. It must preserve Stoquify's existing source-of-truth services and controlled workflows.

2. Critical Path

Shortest safe path to a working internal pilot:

1. Freeze the runtime boundary and prohibited action list.
2. Add agent runtime Prisma models.
3. Create services/agents.
4. Build static read-only tool registry.
5. Build trusted context resolver using existing RBAC and module entitlement.
6. Build evidence binder using snapshots, proof trails, source hashes, evidence grades, blockers, and redactions.
7. Build redaction wrapper using existing redaction policy service.
8. Build deterministic run logger and step logger.
9. Add no-direct-write and no-unsafe-tool tests.
10. Add read-only Command Agent service.
11. Add first UI panel in Daily Digest or Manager Action Center.
12. Run focused tests and existing release gates.
13. Start internal pilot with owner, manager, accountant, finance officer, and stockkeeper roles.

Do not introduce broad AI dependency work before step 9. The runtime boundary must exist before model calls are added.

3. Step-By-Step Implementation Plan

Step 1: Create Design Freeze Artifact

Purpose:

- Make the runtime scope explicit before code begins.

Files or folders:

- docs/agents-runtime/
- optional what-next/agents-runtime/

Implementation tasks:

- Record MVP agents: Command, Cash/Reconciliation, Inventory/Replenishment.
- Record prohibited actions: raw Prisma write, ledger posting, statutory filing, payroll approval, close certification, cash adjustment, stock write-off, permission changes.
- Record MVP read tools.
- Record dependency adoption order.

Dependencies:

- Strategic roadmap report.

Output artifact:

- Agent runtime design freeze document.

Verification:

- Document review by architecture, product, and security.

Acceptance criteria:

- MVP scope is explicit.
- Prohibited actions are explicit.
- First pilot roles are explicit.
- No code work is blocked by ambiguity.

Risks:

- Scope expands into generic chatbot work.

Rollback or stop condition:

- Stop if stakeholders request direct action execution before runtime gates exist.

Step 2: Add Runtime Schema

Purpose:

- Persist agent governance, runs, tool definitions, evidence links, feedback, cost, and incidents.

Files or folders:

- prisma/schema.prisma
- new Prisma migration

Implementation tasks:

- Add AgentDefinition.
- Add AgentSkillDefinition.
- Add AgentToolDefinition.
- Add AgentRun.
- Add AgentStep.
- Add AgentEvidenceLink.

- Add AgentFeedback.
- Add AgentCostLedger.
- Add AgentPolicyIncident.
- Defer AgentActionDraft and AgentApproval until read-and-draft phases unless placeholders are needed.

Dependencies:

- Design freeze.

Output artifact:

- Prisma schema update and migration.

Verification:

```
npm run prisma:validate
npm run typecheck
```

Acceptance criteria:

- Schema validates.
- Models are tenant-scoped where required.
- Uniqueness exists for agent keys, skill versions, tool keys, run correlation IDs, and idempotency-related fields where applicable.

Risks:

- Overly broad schema creates premature complexity.

Rollback or stop condition:

- Stop if schema requires business data mutation paths in Phase 1.

Step 3: Create services/agents Foundation

Purpose:

- Establish the service-owned runtime boundary.

Files or folders:

- services/agents/agent-contracts.ts
- services/agents/agent-context.service.ts
- services/agents/agent-tool-registry.service.ts
- services/agents/agent-evidence.service.ts
- services/agents/agent-redaction.service.ts
- services/agents/agent-runner.service.ts
- services/agents/agent-policy.service.ts
- services/agents/___tests___/

Implementation tasks:

- Define shared types and risk levels.
- Define tool categories.
- Define run and step creation contracts.
- Define evidence link contract.
- Define prohibited tool policy.

Dependencies:

- Runtime schema.
- Existing RBAC, module entitlement, redaction, evidence, snapshot, signal, and action queue services.

Output artifact:

- Agent service foundation.

Verification:

```
npm run typecheck
npm test -- --runInBand services/agents
```

Acceptance criteria:

- Service compiles.
- Tests can instantiate registry, context resolver, and policy checks.
- No UI or model dependency is required yet.

Risks:

- Service imports client-only modules.
- Service bypasses existing protection patterns.

Rollback or stop condition:

- Stop if services/agents needs direct Prisma business writes.

Step 4: Build Trusted Context Resolver

Purpose:

- Resolve the exact actor, tenant, permissions, role, locale, currency, route, module access, and time window before tools or model output.

Files or folders:

- services/agents/agent-context.service.ts
- tests under services/agents/__tests__/agent-context.service.test.ts

Implementation tasks:

- Reuse requireRbacContext or equivalent server-side RBAC context.
- Evaluate module entitlement for source modules.
- Normalize locale, currency, organizationId, userId, route source, and date window.
- Return only safe context for downstream tools.

Dependencies:

- lib/security/rbac.ts
- services/modules/module-entitlement.service.ts
- existing i18n/organization settings helpers as needed

Output artifact:

- AgentExecutionContext.

Verification:

```
npm test -- --runInBand services/agents/__tests__/agent-context.service.test.ts
```

Acceptance criteria:

- Context fails closed without tenant.
- Context differs by permissions.
- Module unavailable state is represented.
- No raw session claims are trusted without RBAC resolution.

Risks:

- Permission bypass.

Rollback or stop condition:

- Stop if context cannot be resolved through existing RBAC.

Step 5: Build Read-Only Tool Registry

Purpose:

- Expose only service-owned read tools in Phase 1.

Files or folders:

- services/agents/agent-tool-registry.service.ts
- services/agents/tools/command-tools.ts
- services/agents/tools/proof-trail-tools.ts
- tests under services/agents/___tests___/agent-tool-registry.service.test.ts

Initial tools:

- readTenantOperatingSnapshot
- readBusinessSignals
- readActionQueue
- readProofTrail
- readPaymentTruthSnapshot
- readPaymentReconciliationWorkbench
- readInventoryCashSnapshot

Implementation tasks:

- Define tool key, description, owner service, module slug, required permission, risk level, input schema, output schema, and evidence behavior.
- Deny raw Prisma, ledger, statutory, payroll approval, close certification, cash adjustment, stock mutation, and permission-change tools.
- Implement registry filtering by actor permissions and module entitlement.

Dependencies:

- context resolver
- services/snapshots
- services/signals
- services/evidence
- services/payments

Output artifact:

- Static MVP read-only registry.

Verification:

```
npm test -- --runInBand services/agents/___tests___/agent-tool-registry.service.test.ts
```

Acceptance criteria:

- No write tool is registered.
- Every tool has permission and module slug.
- Registry filters tools by user context.
- Prohibited tool keys are rejected.

Risks:

- Tool registry becomes a hidden bypass.

Rollback or stop condition:

- Stop if a tool requires direct business mutation.

Step 6: Build Evidence Binder

Purpose:

- Ensure every output can cite what data it came from.

Files or folders:

- services/agents/agent-evidence.service.ts
- tests under services/agents/___tests___/agent-evidence.service.test.ts

Implementation tasks:

- Normalize evidence grade, freshness, source hash, source modules, blockers, redactions, and proof subjects.
- Link AgentRun and AgentStep to AgentEvidenceLink.
- Mark unavailable evidence explicitly.

Dependencies:

- services/evidence/evidence-contracts.ts
- services/snapshots/snapshot-contracts.ts
- proof trail service

Output artifact:

- Evidence binding API.

Verification:

```
npm test -- --runInBand services/agents/___tests___/agent-evidence.service.test.ts
```

Acceptance criteria:

- Evidence link includes source module and evidence grade.
- Stale/blocked/partial states are preserved.
- Redaction count is preserved.
- Missing evidence cannot be presented as proof.

Risks:

- Recommendations without evidence.

Rollback or stop condition:

- Stop if evidence cannot be attached to MVP outputs.

Step 7: Build Redaction Wrapper

Purpose:

- Prevent sensitive data from entering prompts or final output.

Files or folders:

- services/agents/agent-redaction.service.ts
- tests under services/agents/___tests___/agent-redaction.service.test.ts

Implementation tasks:

- Reuse services/security/redaction-policy.service.ts.
- Apply redaction before model context construction.
- Apply redaction again before final answer rendering.
- Return safe redaction notices.

Dependencies:

- existing redaction policy service
- module entitlement decisions

- actor permissions

Output artifact:

- Safe prompt context builder.

Verification:

```
npm test -- --runInBand services/agents/__tests__/agent-redaction.service.test.ts
```

Acceptance criteria:

- Payroll person amounts, supplier bank details, provider references, suspense details, fiscal payloads, compliance payloads, audit context, close evidence, and export data follow policy.
- Redacted values cannot be recovered from run summaries.

Risks:

- Sensitive data leakage.

Rollback or stop condition:

- Stop if traces or prompts would store unredacted sensitive fields.

Step 8: Build Run Logger And Deterministic Runner

Purpose:

- Record execution before adding model calls.

Files or folders:

- services/agents/agent-runner.service.ts
- tests under services/agents/__tests__/agent-runner.service.test.ts

Implementation tasks:

- Create AgentRun.
- Create AgentStep.
- Attach evidence links.
- Record safe summaries only.
- Record policy incidents.
- Return deterministic summary from tools without LLM involvement.

Dependencies:

- runtime schema
- tool registry
- evidence binder
- redaction wrapper

Output artifact:

- Non-LLM runtime pilot.

Verification:

```
npm test -- --runInBand services/agents/__tests__/agent-runner.service.test.ts
```

Acceptance criteria:

- A run can be started, stepped, completed, failed, or blocked.
- Evidence links attach to steps.
- Unsafe tool attempt creates policy incident.
- Safe summaries do not expose redacted fields.

Risks:

- Observability too late.

Rollback or stop condition:

- Stop if run logging cannot remain tenant-scoped.

Step 9: Add Boundary Gates**Purpose:**

- Make runtime safety enforceable.

Files or folders:

- scripts/agent-tool-registry-gate.js
- scripts/agent-prohibited-action-gate.js
- optional scripts/agent-redaction-trace-gate.js
- package.json

Implementation tasks:

- Add script that fails if prohibited tool names are registered.
- Add script that fails if raw Prisma or direct write tools appear in services/agents.
- Add package scripts after initial test stability.

Dependencies:

- tool registry

Output artifact:

- Agent safety gates.

Verification:

```
npm run service:boundary:fail
npm run typecheck
npm run lint
```

Acceptance criteria:

- Prohibited tool gate fails when denylisted tool keys are added.
- Gate can be included in policy:gates later.

Risks:

- Gates too broad and noisy.

Rollback or stop condition:

- Keep as report-only script until signal is reliable.

Step 10: Implement Read-Only Command Agent**Purpose:**

- Deliver first daily-value internal pilot.

Files or folders:

- services/agents/command-agent.service.ts
- services/agents/skills/role-daily-brief.skill.ts
- components/agents/AgentCommandPanel.tsx
- first integration route, preferably Daily Digest or Manager Action Center

Implementation tasks:

- Use read-only tools to create a role-aware daily brief.
- Include evidence grade, freshness, source modules, redactions, and blockers.
- Add feedback capture.
- Keep direct execution disabled.

Dependencies:

- Phase 1 runtime
- optional AI SDK after no-direct-write tests are green

Output artifact:

- Internal read-only Command Agent pilot.

Verification:

```
npm run typecheck
npm run lint
npm run workflow:assurance:runtime-check
npm run report:trust:export:gate
npm test -- --runInBand services/agents
```

Acceptance criteria:

- Agent answers "what matters today" from allowed context only.
- Output changes by role and permission.
- Output includes evidence and limitations.
- Agent refuses direct action requests.

Risks:

- Users expect execution too early.

Rollback or stop condition:

- Hide panel with feature flag if output is stale, unsafe, or not useful.

4. Work Packages And Tickets

Phase 0: Readiness And Design Freeze

Ticket 0.1: Create ADR for agent runtime boundary

Owner: Architecture

Output: design freeze doc

Done when: MVP scope, prohibited actions, pilot roles, and dependency order are approved.

Ticket 0.2: Define agent risk taxonomy

Owner: Security and Product

Output: risk matrix

Done when: read-only, draft, low-risk action, sensitive action, and prohibited are defined.

Ticket 0.3: Define MVP tool list

Owner: Architecture

Output: read-only tool catalog

Done when: each tool has owner service, permission, module slug, evidence behavior, and risk level.

Phase 1: Shared Runtime Foundation

Ticket 1.1: Add runtime Prisma models

Owner: Platform

Output: schema and migration

Done when: npm run prisma:validate passes.

Ticket 1.2: Add services/agents contracts

Owner: Platform

Output: agent-contracts.ts

Done when: risk levels, run states, step states, tool definitions, and evidence link types compile.

Ticket 1.3: Implement context resolver

Owner: Security/Platform

Output: agent-context.service.ts

Done when: tenant, permissions, module entitlement, route, locale, and timeframe are resolved safely.

Ticket 1.4: Implement tool registry

Owner: Platform

Output: agent-tool-registry.service.ts

Done when: read-only tools filter by context and prohibited tools are rejected.

Ticket 1.5: Implement evidence binder

Owner: Platform

Output: agent-evidence.service.ts

Done when: source hash, evidence grade, freshness, blockers, redactions, and proof subjects attach to steps.

Ticket 1.6: Implement redaction wrapper

Owner: Security

Output: agent-redaction.service.ts

Done when: sensitive categories are redacted before prompt context.

Ticket 1.7: Implement deterministic runner

Owner: Platform

Output: agent-runner.service.ts

Done when: runs and steps record without model calls.

Ticket 1.8: Add safety tests and gates

Owner: QA/Security

Output: focused tests and optional scripts

Done when: no-direct-write tests fail on prohibited tool registration.

Phase 2: Read-Only Command Agent

Ticket 2.1: Add Command Agent service

Owner: Product Engineering

Output: command-agent.service.ts

Done when: role daily brief uses read-only tools and evidence.

Ticket 2.2: Add Command Agent prompt/skill template

Owner: Product/Platform

Output: versioned skill definition

Done when: prompt is evidence-constrained and refuses execution.

Ticket 2.3: Add AgentCommandPanel

Owner: Frontend

Output: embedded daily brief UI

Done when: panel renders evidence, freshness, redactions, and feedback controls.

Ticket 2.4: Integrate first route

Owner: Frontend

Output: Daily Digest or Manager Action Center integration

Done when: internal users can view the brief.

Phase 3: Cash/Reconciliation Agent

Ticket 3.1: Wrap payment truth and reconciliation tools

Owner: Finance Engineering

Output: payment agent tools

Done when: read tools return redacted, evidence-bound data.

Ticket 3.2: Add AgentActionDraft

Owner: Platform

Output: draft schema and service

Done when: draft suggestions store risk, permission, evidence, and payload hash.

Ticket 3.3: Add reconciliation draft card

Owner: Frontend

Output: draft UI in finance reconciliation

Done when: suggestions cannot execute directly.

Phase 4: Inventory/Replenishment Agent

Ticket 4.1: Wrap inventory cash and stock-to-cash tools

Owner: Inventory Engineering

Output: inventory agent tools

Done when: stock risk explanation uses evidence and source hash.

Ticket 4.2: Add replenishment draft types

Owner: Inventory/Product

Output: reorder, transfer review, stock count review drafts

Done when: no stock mutation occurs.

Ticket 4.3: Add inventory assistant panel

Owner: Frontend

Output: Inventory/Replenishment panel

Done when: panel shows stock risk, evidence, and draft recommendations.

Phase 5: Approval, Idempotency, Low-Risk Actions

Ticket 5.1: Add AgentApproval

Owner: Platform/Security

Output: approval model and service

Done when: approval replay revalidates permission, entitlement, fresh auth, and input hash.

Ticket 5.2: Add low-risk action executor

Owner: Platform

Output: controlled service execution path

Done when: only approved low-risk actions execute through protected services.

Ticket 5.3: Add approval timeline UI

Owner: Frontend

Output: approval state component

Done when: requested, approved, rejected, executed, failed, and expired states are visible.

Phase 6: Observability And Trust

Ticket 6.1: Add agent run admin surface

Owner: Platform/Frontend

Output: agent-runs route

Done when: runs, steps, costs, feedback, and incidents are inspectable.

Ticket 6.2: Add redacted trace integration

Owner: Platform/Security

Output: Langfuse, Helicone, or equivalent proof-of-concept

Done when: no sensitive payloads are exported.

Ticket 6.3: Add trust monitor

Owner: Product/Platform

Output: stale, unsafe, low-acceptance, and cost alerts

Done when: incidents and trend metrics are visible.

Phase 7: Expansion

Ticket 7.1: Purchasing/AP Agent design

Ticket 7.2: Close/Compliance Agent design

Ticket 7.3: Payroll Readiness Agent design

Ticket 7.4: Customer Success/Adoption Agent design

Done when:

- each expansion agent reuses the shared runtime and has no separate framework.

5. Agent Realization Plan

Command Agent

First useful version:

- Read-only daily brief answering: "What matters today and why?"

Tools needed:

- readTenantOperatingSnapshot
- readBusinessSignals
- readActionQueue
- readProofTrail
- readCloseReadinessSnapshot

Services to wrap:

- services/snapshots
- services/signals
- services/evidence

Permissions:

- dashboard read plus each signal's required permission.

Evidence requirements:

- evidence grade, source modules, source hash, freshness, blockers, redactions.

Redaction rules:

- apply all relevant redaction policies before prompt and output.

UI entry point:

- Daily Digest first, Manager Action Center second, Owner War Room third.

Prohibited actions:

- no assignment, resolution, approval, or mutation in first version.

Tests:

- context by role
- signal filtering
- stale snapshot handling
- redaction
- no direct action

Success metrics:

- internal weekly active users
- helpful feedback
- action queue engagement
- stale answer rate
- redaction correctness

Cash/Reconciliation Agent**First useful version:**

- Read-and-draft triage for duplicate references, suspense, pending payments, and cash variance.

Tools needed:

- readPaymentTruthSnapshot
- readPaymentReconciliationWorkbench
- readCashPaymentHistory
- readProofTrail
- draftReconciliationMatchSuggestion
- draftSuspenseClassification

Services to wrap:

- services/payments
- payment truth snapshot
- proof trail service
- finance/cash workbench services

Permissions:

- payments.reconciliation.read
- finance payment permissions
- stricter suspense detail permissions.

Evidence requirements:

- payment transaction or reconciliation proof subject, provider reference coverage, suspense amount, evidence grade, redaction state.

Redaction rules:

- mask provider references when permission is insufficient.
- redact suspense details when permission is insufficient.

UI entry point:

- Finance Reconciliation first, Cash Command second.

Prohibited actions:

- no cash adjustment
- no suspense posting
- no ledger posting
- no provider mutation

Tests:

- false direct action request refusal
- draft-only suspense
- redacted provider references
- payment cash truth gate

Success metrics:

- suspense exposure reduction
- duplicate reference triage time
- accepted match drafts
- false suggestion rate

Inventory/Replenishment Agent

First useful version:

- Read-and-draft stock risk and replenishment plan.

Tools needed:

- readInventoryCashSnapshot
- readStockToCashFlow
- readInventoryMovements
- readItemSupplierLinks
- draftReplenishmentSuggestion
- draftTransferReview
- draftStockCountReview

Services to wrap:

- services/snapshots/inventory-cash-snapshot.service.ts
- services/stock-to-cash
- inventory movement hooks/services
- purchasing/item supplier read models

Permissions:

- inventory read permissions
- purchasing read permissions for supplier context
- draft permissions later.

Evidence requirements:

- inventory levels, movement history, purchase order context, source hash, negative/zero stock blockers.

Redaction rules:

- protect supplier/payment-sensitive data if exposed.

UI entry point:

- Inventory overview first, Stock-to-Cash second.

Prohibited actions:

- no stock adjustment
- no write-off
- no stock count approval
- no transfer execution
- no purchase order approval

Tests:

- negative stock explanation
- zero stock explanation
- draft remains non-executing
- inventory boundary gate

Success metrics:

- negative stock blocker reduction

- stockout reduction
- replenishment draft acceptance
- dead stock exposure visibility

6. Skill Realization Plan

Trusted Context Resolver

Location:

- services/agents/agent-context.service.ts

Inputs:

- actor/session context, route, requested agent, date range, locale.

Outputs:

- tenant-scoped AgentExecutionContext.

Dependencies:

- RBAC, module entitlement, organization settings.

Tests:

- no tenant fails closed
- permission changes affect tools
- module unavailable affects tools

Done:

- all agent runs start from trusted context.

Permission And Entitlement Guard

Location:

- services/agents/agent-tool-registry.service.ts
- services/agents/agent-policy.service.ts

Inputs:

- context and tool definitions.

Outputs:

- allowed tool list and policy denials.

Dependencies:

- RBAC permission helpers and module entitlement service.

Tests:

- unknown permission denied
- unavailable module denied
- wildcard does not bypass entitlement

Done:

- model cannot see disallowed tools.

Evidence-Grounded Retrieval

Location:

- services/agents/agent-evidence.service.ts

Inputs:

- snapshot results, proof trails, business signals, workbench outputs.

Outputs:

- evidence-bound context blocks and AgentEvidenceLink rows.

Dependencies:

- evidence contracts, snapshot contracts, proof trail service.

Tests:

- stale, blocked, partial, and redacted evidence states preserved.

Done:

- every recommendation has evidence or explicit no-evidence state.

Redaction And Safe Output

Location:

- services/agents/agent-redaction.service.ts

Inputs:

- raw service output, context, field categories.

Outputs:

- prompt-safe and output-safe objects.

Dependencies:

- redaction policy service.

Tests:

- sensitive categories redacted before prompt.

Done:

- no sensitive payload reaches prompt, run summary, trace, or UI without policy approval.

Freshness Evaluator

Location:

- services/agents/skills/freshness-evaluator.skill.ts

Inputs:

- snapshot freshness, generatedAt, sourceMaxUpdatedAt, max age.

Outputs:

- fresh, stale, partial, blocked, failed, or empty state.

Dependencies:

- snapshot contracts.

Tests:

- stale data produces caution state.

Done:

- agents cannot present stale data as current.

Action Draft Builder

Location:

- services/agents/agent-action-draft.service.ts

Inputs:

- recommendation, target records, evidence, required permission, risk.

Outputs:

- AgentActionDraft.

Dependencies:

- tool registry, evidence binder, prohibited action policy.

Tests:

- prohibited draft types fail.

Done:

- drafts can be displayed but not executed without approval path.

Approval Router

Location:

- services/agents/agent-approval.service.ts

Inputs:

- draft, actor, risk, permission, module entitlement, fresh auth.

Outputs:

- approval request and replay validation result.

Dependencies:

- fresh auth, RBAC, module entitlement.

Tests:

- permission loss blocks replay.
- stale input hash blocks replay.

Done:

- low-risk execution cannot bypass approval policy.

Agent Trust Monitor

Location:

- services/agents/agent-observability.service.ts

Inputs:

- runs, feedback, policy incidents, costs, stale states.

Outputs:

- trust metrics and incidents.

Dependencies:

- runtime tables and optional observability provider.

Tests:

- unsafe attempts create incidents.

Done:

- product and security can inspect trust and risk.

7. Data Model Execution Plan

AgentDefinition

Introduce:

- Phase 1.

Why:

- Prevent hidden ad hoc agents.

Key fields:

- id, key, name, description, ownerModuleSlug, status, rolloutMode, riskLevel, defaultModelPolicy, allowedToolKeys, allowedSkillKeys, createdAt, updatedAt.

Relationships:

- one-to-many with AgentRun.

Indexes:

- unique key.

Migration risk:

- low.

Tests:

- duplicate key rejected.

AgentSkillDefinition

Introduce:

- Phase 1.

Why:

- Version Stoquify-native skills.

Key fields:

- id, key, version, domain, promptHash, ownerModuleSlug, requiredPermissions, redactionCategories, status.

Relationships:

- referenced by AgentDefinition through key list or join table later.

Indexes:

- unique key plus version.

Migration risk:

- low.

Tests:

- duplicate key/version rejected.

AgentToolDefinition

Introduce:

- Phase 1.

Why:

- Govern model-visible tools.

Key fields:

- id, key, ownerService, moduleSlug, requiredPermission, riskLevel, toolType, inputSchemaHash, outputSchemaHash, approvalPolicy, idempotencyMode, status.

Relationships:

- referenced by AgentStep.

Indexes:

- unique key.

Migration risk:

- medium if risk and tool-type enums are too narrow.

Tests:

- prohibited tool key rejected by registry gate.

AgentRun

Introduce:

- Phase 1.

Why:

- Trace every agent execution.

Key fields:

- id, organizationId, actorId, agentKey, status, sourceRoute, locale, currency, periodStart, periodEnd, correlationId, startedAt, completedAt, failureCode.

Relationships:

- has many steps, evidence links, feedback, costs, policy incidents.

Indexes:

- organizationId plus createdAt
- actorId plus createdAt
- correlationId unique

Migration risk:

- medium due tenant volume.

Tests:

- run must be tenant-scoped.

AgentStep

Introduce:

- Phase 1.

Why:

- Debug and audit tool calls and model steps.

Key fields:

- id, runId, stepNumber, kind, toolKey, status, inputHash, outputHash, safeSummary, startedAt, completedAt, errorCode.

Relationships:

- belongs to run.

Indexes:

- runId plus stepNumber unique.

Migration risk:

- low.

Tests:

- ordered steps recorded.

AgentEvidenceLink

Introduce:

- Phase 1.

Why:

- Ensure recommendations stay tied to proof.

Key fields:

- id, runId, stepId, subjectType, subjectId, sourceModule, sourceTable, sourceHash, evidenceGrade, freshness, blockerCount, redactionCount.

Relationships:

- belongs to run and optionally step.

Indexes:

- runId

- subjectType plus subjectId

Migration risk:

- low.

Tests:

- evidence link required for recommendations.

AgentActionDraft

Introduce:

- Phase 3.

Why:

- Store recommendations that may become controlled actions.

Key fields:

- id, runId, serviceActionKey, status, riskLevel, requiredPermission, moduleSlug, visibleSummary, payloadHash, idempotencyKey, expiresAt.

Relationships:

- belongs to run.
- has approval later.

Indexes:

- runId
- idempotencyKey unique where present.

Migration risk:

- medium.

Tests:

- prohibited action draft rejected.

AgentApproval

Introduce:

- Phase 5.

Why:

- Gate low-risk execution and approval replay.

Key fields:

- id, draftId, requestedById, approvedById, status, approvalPolicy, freshAuthRequired, inputHash, requestedAt, approvedAt, rejectedAt, expiresAt.

Relationships:

- belongs to draft.

Indexes:

- draftId
- approvedById plus approvedAt

Migration risk:

- medium.

Tests:

- replay fails on input hash mismatch.

AgentFeedback

Introduce:

- Phase 1 or 2.

Why:

- Learn whether output is useful, wrong, stale, or unsafe.

Key fields:

- id, runId, actorId, helpful, accepted, rejectedReason, staleAnswer, wrongAnswer, unsafeAttempt, correctionText.

Indexes:

- runId
- actorId plus createdAt

Migration risk:

- low.

Tests:

- feedback cannot cross tenant.

AgentCostLedger

Introduce:

- Phase 1 basic, Phase 6 expanded.

Why:

- Track cost by tenant, agent, model, and module.

Key fields:

- id, runId, organizationId, modelProvider, modelName, promptTokens, completionTokens, estimatedCost, currency, budgetBucket.

Indexes:

- organizationId plus createdAt
- modelProvider plus modelName

Migration risk:

- low.

Tests:

- cost row tenant-scoped.

AgentPolicyIncident

Introduce:

- Phase 1.

Why:

- Record unsafe or blocked attempts.

Key fields:

- id, runId, organizationId, actorId, incidentType, severity, policyKey, blockedToolKey, safeSummary, status, resolvedById.

Indexes:

- organizationId plus severity
- runId

Migration risk:

- low.

Tests:

- prohibited tool attempt creates incident.

8. UI Execution Plan

AgentCommandPanel

Route:

- Daily Digest first.

Purpose:

- Show role-aware daily brief.

Definition of done:

- Displays summary, evidence grade, freshness, redaction notice, source modules, and feedback controls.

AgentModuleAssistantPanel

Route:

- Finance Reconciliation, Cash Command, Inventory, Stock-to-Cash later.

Purpose:

- Contextual explanation panel.

Definition of done:

- Can explain current module state without direct action execution.

AgentEvidenceDrawer

Route:

- Any panel with proof subjects.

Purpose:

- Extend proof trail UI with agent run context.

Definition of done:

- Opens evidence nodes, edges, blockers, redactions, and agent evidence links.

AgentActionDraftCard

Route:

- Finance Reconciliation and Inventory after Phase 3/4.

Purpose:

- Show draft suggestion and risk state.

Definition of done:

- Cannot execute without approval path.

AgentApprovalTimeline

Route:

- Action draft surfaces after Phase 5.

Purpose:

- Show approval lifecycle.

Definition of done:

- Shows requested, pending, approved, rejected, expired, executed, failed.

AgentTrustNotice

Route:

- All agent outputs.

Purpose:

- Show evidence, freshness, redaction, and limitation status.

Definition of done:

- No answer appears without trust state.

AgentFeedbackControls

Route:

- All agent outputs.

Purpose:

- Capture helpful, wrong, stale, unsafe, accepted, rejected.

Definition of done:

- Feedback persists to AgentFeedback.

Agent Run/Admin Surface

Route:

- app/[locale]/(dashboard)/dashboard/agent-runs

Purpose:

- Inspect runs, steps, costs, evidence, feedback, and incidents.

Definition of done:

- Security/admin roles can inspect runtime trust state.

9. Technology Adoption Sequence

Phase 0:

- No new AI dependencies.
- Finalize tool boundaries and risk taxonomy.

Phase 1:

- No model dependency required.
- Build deterministic runtime first.
- Design tool definitions to be MCP-compatible, but do not expose full MCP server.

Phase 2:

- Introduce AI SDK or equivalent TypeScript-native tool loop after no-direct-write tests are green.
- Use read-only tools only.

Phase 3 and 4:

- Continue with AI SDK or equivalent.
- Do not add durable execution yet unless drafts require waiting workflows.

Phase 5:

- Evaluate Inngest or Trigger.dev for approval wait, retry, scheduled digests, and resume.
- Choose one durable execution layer, not both.

Phase 6:

- Add Langfuse, Helicone, or equivalent observability after redaction-safe trace policy exists.
- Consider LiteLLM only when multi-provider routing and budgets become real needs.

Do not introduce:

- multiple agent frameworks
- Python-first runtime
- broad MCP server
- observability traces before redaction tests
- durable execution before idempotency
- action execution tools before approval replay validation

10. Verification Plan

Phase 0:

- Document review only.

Phase 1:

```
npm run prisma:validate
npm run typecheck
npm run lint
npm run service:boundary:fail
npm test -- --runInBand services/agents
```

Phase 2:

```
npm run typecheck
npm run lint
npm run workflow:assurance:runtime-check
npm run report:trust:export:gate
npm test -- --runInBand services/agents
```

Phase 3:

```
npm run payment:cash-truth:gate
npm run ledger:close-truth:gate
npm run report:trust:export:gate
npm test -- --runInBand services/agents services/payments
```

Phase 4:

```
npm run inventory:boundary:fail
npm run service:boundary:fail
npm run ledger:close-truth:gate
npm test -- --runInBand services/agents
```

Phase 5:

```
npm run service:boundary:fail
npm run workflow:assurance:runtime-check
npm test -- --runInBand services/agents
```

Phase 6:

```
npm run report:trust:export:gate
npm run release:secrets:preflight
```

```
npm run ci:release:gate
npm test -- --runInBand services/agents
```

Full readiness:

```
npm run verify:repo
```

11. Risk Control Plan

Risk	Control	Verification	Stop Condition
Direct database mutation	no raw Prisma tools, tool denylist	prohibited action test	any agent tool writes directly to Prisma
Permission bypass	context resolver plus RBAC plus module entitlement	context and registry tests	actor sees unauthorized tool
Redaction failure	redaction before prompt and output	redaction tests	sensitive value appears in context
Stale recommendations	freshness evaluator	stale snapshot tests	stale data shown as current
False reconciliation matches	draft-only, confidence, human review	payment tests	agent posts or resolves match
Unsafe approval replay	input hash, revalidation, fresh auth	approval tests	approval executes changed input
Payroll/statutory leakage	defer agents, redaction, provenance	statutory/payroll gates later	non-authoritative output appears as final
Cost spikes	cost ledger and model policy	cost monitor in Phase 6	unbounded recurring model calls
Generic chatbot drift	embedded panels only	product review	standalone chatbot becomes MVP
Dependency sprawl	one runtime, one durable layer	ADR review	module-specific frameworks appear

12. 30/60/90-Day Practical Execution Schedule

Week 1

Engineering:

- create design freeze
- define runtime schema draft
- define prohibited tool denylist

Product/design:

- define first daily brief scenarios
- define pilot roles

Security/control:

- approve risk taxonomy
- approve no-direct-write policy

Deliverables:

- design freeze and MVP tool list

Decision gate:

- approve Phase 1 implementation

Week 2

Engineering:

- add services/agents/agent-contracts.ts
- add context resolver skeleton
- add tool registry skeleton
- add first no-direct-write test

Product/design:

- define Command Agent answer shape

Security/control:

- review context resolver behavior

Deliverables:

- compile-ready service skeleton

Decision gate:

- approve runtime schema migration

Week 3**Engineering:**

- add Prisma runtime models
- add run and step persistence
- add evidence binder
- add redaction wrapper

Product/design:

- define evidence and freshness display copy

Security/control:

- validate redaction-before-prompt behavior

Deliverables:

- Phase 1 runtime foundation

Decision gate:

- proceed to deterministic runner

Week 4**Engineering:**

- complete deterministic runner
- complete focused tests
- run Phase 1 gates

Product/design:

- define internal pilot feedback taxonomy

Security/control:

- review policy incidents and prohibited tool behavior

Deliverables:

- Phase 1 candidate

Decision gate:

- approve Command Agent build

Weeks 5-6

Engineering:

- add AI SDK or equivalent if Phase 1 gates pass
- implement read-only Command Agent
- implement AgentCommandPanel
- integrate Daily Digest or Manager Action Center

Product/design:

- validate role daily brief with pilot users

Security/control:

- confirm no direct actions

Deliverables:

- internal read-only Command Agent pilot

Decision gate:

- approve limited internal pilot

Weeks 7-8

Engineering:

- add payment/reconciliation read tools
- add draft schema design
- add finance assistant panel prototype

Product/design:

- define reconciliation triage workflow

Security/control:

- review false match and suspense redaction controls

Deliverables:

- Cash/Reconciliation read-and-draft plan

Decision gate:

- approve Phase 3 implementation

Weeks 9-10

Engineering:

- add inventory cash and stock-to-cash read tools
- add replenishment draft schema design
- add inventory assistant panel prototype

Product/design:

- define replenishment suggestion rules

Security/control:

- review no stock mutation policy

Deliverables:

- Inventory/Replenishment read-and-draft plan

Decision gate:

- approve Phase 4 implementation

Weeks 11-12

Engineering:

- implement approval model design
- implement idempotency strategy
- prototype agent run/admin view

Product/design:

- create pilot scorecard

Security/control:

- review approval replay and trace redaction policy

Deliverables:

- Phase 5/6 readiness package

Decision gate:

- continue, pause, or expand based on pilot metrics

13. Definition Of Ready And Done

Definition of ready for any phase:

- source report and phase scope are clear
- prohibited actions are unchanged or explicitly reviewed
- service owner is identified
- RBAC/module entitlement impact is known
- evidence source is known
- redaction category is known
- tests are named
- rollback or stop condition is defined

Definition of done for any phase:

- implementation compiles
- focused tests pass
- relevant release gates pass
- agent output is tenant-scoped
- permissions are enforced
- module entitlement is enforced
- evidence is attached or unavailable state is explicit
- redaction policy is applied
- no prohibited action is possible
- user-facing UI has trust/freshness state
- documentation is updated

14. First Implementation Prompt

Use this prompt for the next Codex run:

Act as a senior Stoquify platform engineer and security architect.

Implement Phase 0 and Phase 1 of the Stoquify agent-runtime execution plan only.

Source plan:

docs/agents-runtime/STOQUIFY_AGENT_RUNTIME_PRACTICAL_EXECUTION_PLAN_2026-07-22.md

Scope:

- Create a Phase 0 design freeze artifact under docs/agents-runtime.
- Add the shared runtime foundation under services/agents.
- Add minimum Prisma models for AgentDefinition, AgentSkillDefinition, AgentToolDefinition, AgentRun, AgentStep, AgentEvidenceLink, AgentFeedback, AgentCostLedger, and AgentPolicyIncident.
- Add a static read-only MVP tool registry.
- Add a trusted context resolver using existing RBAC and module entitlement.
- Add an evidence binder for source hash, evidence grade, freshness, blockers, redactions, and source modules.
- Add a redaction wrapper using existing redaction policies.
- Add a deterministic run logger and step logger.
- Add focused tests for tenant isolation, permission filtering, module entitlement filtering, redaction, evidence binding, and no direct Prisma write tool registration.

Non-goals:

- Do not add visible agent UI.
- Do not add AI SDK yet unless needed only for type contracts.
- Do not add durable execution.
- Do not add observability vendors.
- Do not create action execution tools.
- Do not allow direct Prisma writes, ledger posting, statutory filing, payroll approval, close certification, cash adjustment, stock write-off, or permission changes.
- Do not refactor unrelated modules or fix unrelated lint issues.

Verification:

- npm run prisma:validate
- npm run typecheck
- npm run lint
- npm run service:boundary:fail
- npm test -- --runInBand services/agents

Stop and report if:

- existing schema conventions conflict with the proposed model names
- RBAC/module entitlement cannot be reused safely
- any MVP tool requires direct business mutation
- redaction cannot be applied before prompt context construction

Immediate Recommendation

Begin with Phase 0 and Phase 1 only. Do not install AI SDK or build visible agent UI until the runtime schema, context resolver, read-only tool registry, evidence binder, redaction wrapper, run logger, and no-direct-write tests are in place. This gives Stoquify the safety foundation needed to realize the agents and skills without weakening the product's core trust model.